

Submission 7588	Help	Conference	News	EasyChair
-----------------	------	------------	-------------	-----------

FPL 2025 Submission 7588

Submission information updates are disabled.

For all questions related to your submission you should contact the conference organizers. [Click here to see information about this conference.](#)

All **reviews sent to you** can be found at the bottom of this page.

Submission 7588	
Title	High Throughput Large Integer Multipliers Built on Optimized FPGA DSP Block Primitives
Paper	 (Mar 31, 09:12)
Slides	
Bio	
Track	FPL 2025
Author keywords	DSP Blocks Arithmetic Multipliers
EasyChair keyphrases	dsp blocks (170), large integer multiplication (126), large integer multipliers (95), dsp block (90), base multiplier (85), dsp slice (80), design of integrated circuits and systems (77), higher frequency (70), dsp slices (70), aided design of integrated circuits (69), digital signal processing (63), computer arithmetic arith (63), multiplier size bits (63), high frequency (60), decomposition schemes (60), cascade path (60), resource optimal design of large multipliers (51), maximum frequency (50), schoolbook decomposition (50), dsp utilization (50), computer aided design (47), programmable gate arrays (47), ip based multiplier (47), primitive based design (47), primitive based multipliers (47), integrated circuits and systems pp (46), utilization with different decomposition schemes (46), integer multiplication (40), partial products (40), right shift (40), bit multiplier (40), underlying architecture (40), terms of resource utilization (40), lut utilization (40), dsp multiplier (40), shift and sign extension (40), latency cycles wasted dsps (40), area and performance comparison (40), primitive based large multipliers (40)
Topics	Long paper
Abstract	Large integer multiplication is critical in cryptographic applications such as Fully Homomorphic Encryption (FHE) and RSA where it represents the main step in computing the modular multiplication used to generate keys. Most previous work building such multipliers on FPGAs has overlooked the importance of considering the underlying architecture of the Digital Signal Processing (DSP) Blocks provided in modern FPGAs. These DSP blocks can operate at high frequency but present an irregular input size that must be tiled to create large multipliers. In this paper, we show how a near-optimal base multiplier can be built using modern DSP blocks, then scaled to large sizes, while maintaining resource efficiency and high throughput. We demonstrate the automation of tiling, and establish a new performance standard for large integer multipliers on FPGAs.

Submitted	Mar 20, 12:53
Last update	Mar 31, 09:13
Conflicts of interest	
Author conflicts	Christos Bouganis, Peter Cheung, Zhenman Fang, Nachiket Kapre, Dirk Koch, Jason Lau, Shreejith Shanker (view)

Authors						
First name	Last name	Email	Country	Affiliation	Web page	Corresponding?
Mahmoud	Elshenawy	mahmoud.elshenawy@kaust.edu.sa	Saudi Arabia	King Abdullah University of Science and Technology		✓
Zhiru	Zhang	zhiruz@cornell.edu	United States	Cornell University		✓
Suhaib A.	Fahmy	suhaib.fahmy@kaust.edu.sa	Saudi Arabia	King Abdullah University of Science and Technology		✓

Reviews

Review 1	
<i>Novelty</i>	4: Not much new insights or just an incremental improvement
<i>Writing Quality</i>	3: Needs improvement. Sections make sense, but not as a whole paper.
<i>Overall Evaluation</i>	–1: Weak reject
<i>Strengths</i>	The proposed multipliers require 13% fewer DSP blocks and achieve 73% higher throughput (i.e., clock frequency) compared to existing multipliers.
<i>Weaknesses</i>	The proposed multipliers have an order of magnitude longer latency compared to existing multipliers.
<i>Comments for Authors</i>	This paper studies large integer multiplication, emphasizing the importance of considering the underlying architecture of Digital Signal Processing (DSP) blocks in modern FPGAs. The authors introduce a near-optimal base multiplier design using DSP blocks, scalable to large sizes while maintaining resource efficiency and high throughput. Key contributions include a highly efficient 78×78 base multiplier, an automated tool for building large multipliers from a high-level JSON description, and a comparative analysis showing significant performance improvements. The proposed Primitive-based approach achieves around 73% higher frequency and uses 13% fewer DSP blocks compared to other methods. Table 2 shows that while the proposed multipliers improve throughput by up to 2x compared to HLS-generated multipliers, they also exhibit an order of magnitude longer latency. Applications that can tolerate this extremely long latency are likely to be limited. It is recommended to evaluate the multipliers not only in isolation but also at the application level. In addition to resource usage, frequency, and latency, the evaluation should also include power consumption. Section II–A states, "The Schoolbook decomposition method suffers from quadratic scaling complexity, requiring nine multipliers when inputs are divided into three parts. These partial

products need additional adder and shifter circuits for final result computation." The explanation that shifter circuits are needed is generally incorrect. Since the shift amounts are fixed, it is sufficient to rewire connections, and no logic gates are necessary.

The explanation of the AMD DSP Slice Architecture in Section II-C-2) is difficult to understand. The symbols in Figure 3 should correspond consistently with the symbols used in the text.

I understand that the primary contribution of this work is the 78×78 primitive base multiplier. The generation of larger multipliers is only briefly described in Section III-A-2) Multiplier Generation in about 20 lines, leaving the details unclear.

Review 2

<i>Novelty</i>	4: Not much new insights or just an incremental improvement
<i>Writing Quality</i>	5: Well-written
<i>Overall Evaluation</i>	-1: Weak reject
<i>Strengths</i>	+ quality of the writing + easy to read and to understand
<i>Weaknesses</i>	- tool much less generic compared to previous work as it only allows one tile size - pipeline balancing in their proposed 78×78 multiplier - comparisons with some state of the art methods are missing - large latency
<i>Comments for Authors</i>	The paper presents a tool for the design of large multipliers for AMD Virtex FPGAs. The tool uses a hand-optimized 78×78 multiplier as its base. The large multiplier is then created by using the tiling method, which also considers the Karatsuba method to save resources. The quality of the writing is high and the paper is easy to read and to understand. However, I have the following issues with this work: 1) There is a huge body of work that was built on multiplier tiling, proposing generic methods for tiling arbitrary sets of smaller multipliers (tiles). This work uses a drastically simplified tiling heuristic by using only a single 78×78 multiplier tile (the same limitation I see for [20]). This also limits the possible multiplier sizes (or makes sizes that do not match multiples of 78 less efficient). So, their tool is not a contribution to me. 2) The authors argue that previous work like [25][29] has "a limited max operand size of 64 bits." but this is not true. These are generic tiling methods. In [29], I found results for up to a 1024 bit. However, comparisons are missing here. 3) The proposed 78×78 multiplier tile is well done and sound. However, their architecture (Fig. 6) has a very deep pipeline which increases latency and requires a huge amount of resources (FFs) for pipeline balancing. Most partial products can be computed in parallel but authors decided to compute everything in sequence. Of course, this requires the least post-adders but implementing a few adders in logic would compensate for many pipeline stages (and latency!). The results confirm that their latency is much higher compared to previous work. Detailed comments: * Missing point after the 2nd sentence * Forward reference to equations * Make clear that the & in (2) is the concatenate operation * AMD Vertex specific * Section II-C-3: Specify the FPGA arch for the described cascading * The use of „slice" is ambiguous as it may refer to DSP slices or logic slices. Here, you refer to DSP slices * The „optimal resource" consideration has to be taken carefully as it only handles the DSP

resources and no logic of FF resources. Getting the minimal no of DSPs is a rather simple task.

* Fig. 8: Lines are too thick to be readable, e.g., "0-Karat" is not visible

* TABLE II: Please the information of the actual size. From the text, I guess yours is the 1009/1233-bit multiplier case. In the previous figures you varied this size. What about the size of the other methods?

Review 3

<i>Novelty</i>	4: Not much new insights or just an incremental improvement
<i>Writing Quality</i>	5: Well-written
<i>Overall Evaluation</i>	2: Accept
<i>Strengths</i>	– Large multiplier occupies less hardware resources and operates at higher frequency than the given references – Designing automated tool for generating different width of the multiplier
<i>Weaknesses</i>	– Large multipliers (bitwidth larger than 64-bit) are seldom used, sequential operations are more often used – Design is complicated and thus difficult to follow (thus design automation tool is constructed)
<i>Comments for Authors</i>	This paper presents an IP-tool for automated generation of large (wide) bit-width multipliers. An wide multiplier can be form using multiple AMD DSP primitives. In addition the multilevel Karatsuba optimization is used to further reduce the hardware resources. The Karatsuba multiplier requires the additional LUT logics for addition of partial products and for pipelining registers. According to authors, the results are at least 45%, and up to 73% better frequency than the best-performing multiplier in the HLS and IMpress implementations. Summing up, I recommend the paper to be published. The main possible drawback of the paper is that many similar papers have been publish and it is difficult for me to compare the results and ideas at the limited paper size.

Review 4

<i>Novelty</i>	3: Straightforward application of well-known ideas or techniques
<i>Writing Quality</i>	4: Adequate
<i>Overall Evaluation</i>	–1: Weak reject
<i>Strengths</i>	1) This paper was easy to understand. 2) The lower DSP utilization can make the proposed method suitable when there are resource constraints. 3) Figure 9 provides a great comparison with other implementations while showing the trade off in DSP utilization and frequency as well.
<i>Weaknesses</i>	1) A 78x78 bit multiplier, one of the 3 main contributions of this paper, with 10 cycles of extra delay compared to the existing IP making use of an existing tiling method, lacks in both novelty and performance. 2) The main contributions of the paper are not discussed enough in detail. For example, Figures 5 and 6 that should explain the base multiplier, lack information such as the exact bits taken by each tile for the tiling method, and a more detailed hardware representation of the design without using high level blocks. The generator tool, another contribution of this paper, is discussed very briefly without any detail. On top of that a combination of methods from the background section is simply referenced and not discussed enough. You should try to help the reader understand better by using figures and algorithms. 3) In general, the way this paper is structured is unbalanced. A lot more detail is given to

*Comments
for Authors*

background than in the proposed idea itself. You should try to highlight your contributions a lot more and give less attention to the background section.

This paper proposes a method for implementing large integer multipliers on FPGA by utilizing DSP Slices. A 78x78 multiplier is implemented using DSP primitives, which is then used to build larger multipliers by applying combinations of schoolbook approach and Karatsuba's algorithm. The base multiplier is compared with an existing vendor IP, showing better frequency and DSP utilization, with a small overhead in LUT utilization.

Strong points:

This paper was easy to understand.

The lower DSP utilization can make the proposed method suitable when there are resource constraints.

Figure 9 provides a great comparison with other implementations while showing the trade off in DSP utilization and frequency as well.

Weak points:

A 78x78 bit multiplier, one of the 3 main contributions of this paper, with 10 cycles of extra delay compared to the existing IP making use of an existing tiling method, lacks in both novelty and performance.

The main contributions of the paper are not discussed enough in detail. For example, Figures 5 and 6 that should explain the base multiplier, lack information such as the exact bits taken by each tile for the tiling method, and a more detailed hardware representation of the design without using high level blocks. The generator tool, another contribution of this paper, is discussed very briefly without any detail. On top of that a combination of methods from the background section is simply referenced and not discussed enough. You should try to help the reader understand better by using figures and algorithms.

In general, the way this paper is structured is unbalanced. A lot more detail is given to background than in the proposed idea itself. You should try to highlight your contributions a lot more and give less attention to the background section.